BimanualShift: Arm-Conditioned Residual Skill Transfer from Unimanual Policies to Bimanual Manipulation

Yechen Fan, *Graduate Student Member, IEEE,* Jinhua Ye, *Senior Member, IEEE,* Xianyou Ji, Chenyang Song, Haibin Wu, Gengfeng Zheng, *Senior Member, IEEE,* and Jiafu Wan, *Fellow, IEEE*

***Abstract*—Pretrained unimanual policies encode rich manipulation priors, but transferring them to bimanual tasks remains challenging due to shared-workspace perceptual interference, coupled dual-arm coordination, and limited bimanual demonstrations. We propose BimanualShift, an arm-conditioned residual skill transfer framework that adapts frozen unimanual policies without end-to-end retraining. Arm-conditioned visual projection converts the shared workspace into policy-compatible arm-specific observations, reducing cross-arm interference. Bimanual coordination is modeled as low-dimensional residual modulation over a learnable skill basis, where the residual branch compensates for synchronization, spatial avoidance, and contact deviations. For long-horizon execution, successful prior trajectories are retrieved as confidence-gated, temporally aligned residual contexts rather than directly replayed. Experiments in RLBench2 and on a real dual-arm platform demonstrate improved data efficiency, generalization, and long-horizon robustness under limited bimanual demonstrations.**

***Index Terms*—Bimanual manipulation, residual skill transfer, frozen policy priors, data-efficient learning.**

I. Introduction

BIMANUAL manipulation is a key direction for general embodied intelligence, requiring proficient coordination between two hands. Compared with unimanual systems, such tasks exhibit stronger coupling and significantly higher degrees of freedom, perceptual load, and control complexity, making policy learning substantially more challenging [1].

Manuscript received: May 14, 2026; Revised: July 28, 2026; Accepted: November 2, 2026. This paper was recommended for publication by Editor Julia Borras Sol upon evaluation of the Associate Editor and Reviewers' comments. This work was supported in part by the Fujian Provincial Science and Technology Major Project of China under Grant 2024HZ026020, in part by the Projects of Fujian University Industry-Academic Cooperation under Grant 2022H6016, in part by the Project of the Ministry of Industry and Information Technology under Grant 2024-RL-FZSX-01, in part by the Fujian Young Scientific and Technological Personnel Training under Grant 2025350124, and in part by the Projects of Strategic Emerging Industries and Future Industries of Fuzhou-Xiamen-Quanzhou National Independent Innovation Demonstration Zone. *(Corresponding authors: Jinhua Ye and Gengfeng Zheng.)*

Yechen Fan, Jinhua Ye, Xianyou Ji, and Haibin Wu are with the School of Mechanical Engineering and Automation, Fuzhou University, Fuzhou 350116, China (e-mail: yejinhua@fzu.edu.cn).

Chenyang Song is with the School of Automation Science and Electrical Engineering, Beihang University, Beijing 100191, China.

Gengfeng Zheng is with the Fujian Key Laboratory of Special Intelligent Equipment Safety Measurement and Control, Fujian Special Equipment Inspection and Research Institute, Fuzhou 350008, China (e-mail: 18862889747@163.com).

Jiafu Wan is with the Guangdong Province Key Laboratory of Precision Equipment and Manufacturing Technique, School of Mechanical and Automotive Engineering, South China University of Technology, Guangzhou 510641, China.

Digital Object Identifier (DOI): see top of this page.

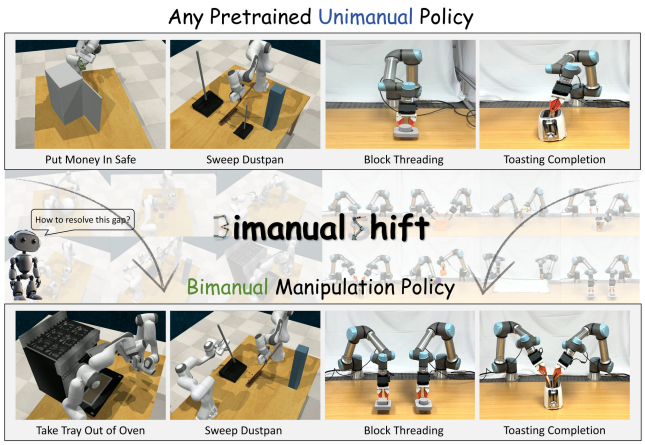


Fig. 1. Overview of BimanualShift. The framework adapts frozen pretrained unimanual policy priors to bimanual manipulation by combining arm-specific perception, residual coordination, and experience-conditioned refinement.

While unimanual manipulation has benefited from large-scale pretrained policies, transferring such priors to bimanual manipulation remains non-trivial. End-to-end bimanual policies, including ACT [2], Diffusion Policy [3], and recent Vision–Language–Action models [4], [5], [6], [7], [8], can model complex behaviors but usually require large amounts of high-quality bimanual demonstrations to cover increased action dimensionality, contact interactions, and temporal synchronization.

A more data-efficient direction is to reuse pretrained unimanual policies as manipulation priors. Recent transfer-based methods such as AnyBimanual [9] show the promise of unimanual-to-bimanual transfer through visual alignment and skill scheduling. However, bimanual manipulation involves complex coordination patterns, role assignments, and physical interactions [10], making direct adaptation to shared workspaces difficult. Holistic or weakly separated inputs can cause target-assignment ambiguity, while independently executed unimanual actions lack the coordination needed for spatial avoidance, force interaction, and temporal synchronization, especially in contact-rich or deformable-object tasks [11]. Existing bimanual imitation-learning and transfer-based methods still face challenges in data efficiency, robustness, and long-horizon generalization [12], and methods such as PerAct^2 [13] and VoxAct-B [14] lack explicit reuse of successful prior executions. These limitations motivate treating unimanual-to-bimanual transfer as residual coordination over frozen unimanual priors rather than direct policy reuse.

To address these challenges, we propose BimanualShift, an arm-conditioned skill transfer framework that adapts frozen

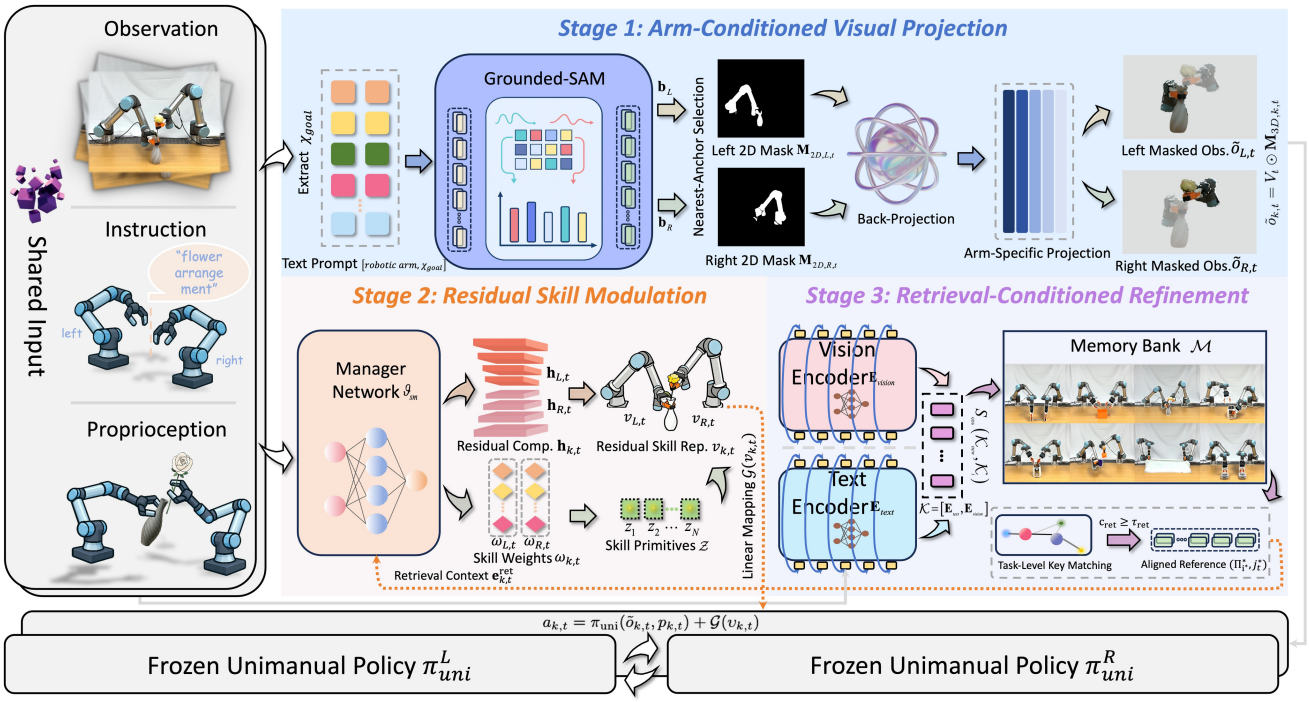


Fig. 2. Technical framework of BimanualShift. Bimanual manipulation is formulated as arm-conditioned residual skill transfer over frozen unimanual policy priors. Visual projection produces policy-compatible arm-specific observations, residual modulation coordinates the frozen policy outputs, and retrieval-conditioned refinement uses task-level keys to retrieve successful executions as confidence-gated contexts for long-horizon correction, without directly replaying their actions.

unimanual policies to bimanual manipulation through residual coordination (Fig. 1). Arm-conditioned visual projection aligns the shared workspace with the observation distribution expected by each unimanual policy, reducing cross-arm perceptual interference before inference. Residual skill modulation then re-parameterizes dual-arm coordination as lightweight residual actions over frozen policy outputs, compensating for synchronization, spatial avoidance, and contact variations. Retrieval-conditioned residual refinement further uses successful prior executions as residual contexts rather than replay trajectories, enabling local correction of accumulated errors while keeping the pretrained policies unchanged.

In summary, the main contributions are as follows:

- We formulate unimanual-to-bimanual transfer as arm-conditioned residual skill transfer, reusing frozen unimanual policies and learning only lightweight coordination residuals.
- We propose arm-conditioned visual projection to transform shared bimanual observations into policy-compatible arm-specific inputs, reducing cross-arm perceptual interference.
- We introduce residual skill modulation with retrieval-conditioned refinement, which coordinates the frozen policy outputs and reuses successful executions as temporally aligned contexts for long-horizon correction.

II. Related Work

A. Foundation Models for Robotic Manipulation

Large-scale pretrained models and imitation policies have substantially advanced robotic manipulation. End-to-end methods, such as ACT [2] and Diffusion Policy [3], map visual observations directly to continuous actions. Vision–Language–Action models, including the π series [4], [5], [6], Octo [7], and OpenVLA [8], further improve generalization across robot embodiments and language-conditioned tasks. Video-based methods include VPP [15], which uses predicted visual dynamics, and TAC [16], which extracts tool-centric representations from human demonstration videos for policy learning. VLM-based methods further connect multimodal reasoning with robot execution. ManipLLM [17] predicts object-centric contacts and end-effector directions, while OmniManip [18] translates interaction primitives into spatial planning constraints. Unlike these methods, which generate complete actions or task-level constraints, BimanualShift reuses frozen unimanual priors and learns lightweight action-space residuals for dual-arm coordination, avoiding full bimanual policy training.

B. Skill Transfer and Residual Adaptation in Robotics

Reusing learned skills across tasks, environments, and embodiments is important for data-efficient robot learning. Prior work studies visual or latent alignment for skill transfer [19], [20] and bimanual learning from limited demonstrations [10], [12]. Recent unimanual-to-bimanual methods include TwinVLA [21] and EnergyAction [22], which compose unimanual policies at the model and policy-distribution levels, respectively, and AnyBimanual [9], which combines arm-specific visual alignment with dynamic skill scheduling. In contrast, BimanualShift uses arm-conditioned projections to suppress cross-arm interference, preserves frozen unimanual policy outputs, and learns additive coordination residuals, with successful executions retrieved for long-horizon correction.

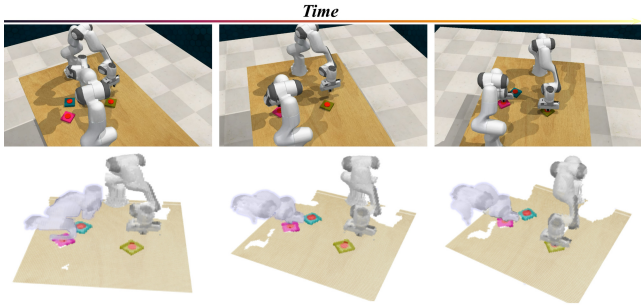


Fig. 3. Semantic voxel scenes for Push Two Buttons. Original RLBench views are shown above, with arm-conditioned projections below that retain the acting arm and task-relevant workspace while suppressing interference.

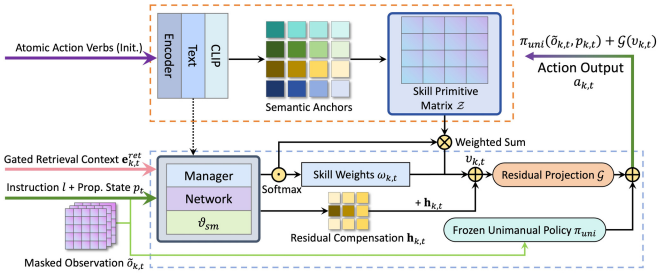


Fig. 4. Residual skill modulation. The Manager Network ϑ_{sm} predicts $\boldsymbol{\omega}_{k,t}$ and $\mathbf{h}_{k,t}$ from arm-specific visual features, proprioception, language, and the gated retrieval context $\mathbf{e}^{\text{ret}}_{k,t}$. The resulting residual $\mathcal{G}(\upsilon_{k,t})$ is added to the frozen unimanual policy output.

Residual policy learning adds lightweight corrections to pretrained or fixed policies. ResiP [23] learns per-timestep residuals through reinforcement learning over a frozen action-chunked policy. Retrieval-augmented methods reuse prior experience: RAEA [24] integrates retrieved policies through cross-attention, while DINOBot [25] retrieves and aligns with a visually similar demonstration before replaying its trajectory. BimanualShift instead learns coordination residuals from bimanual demonstrations and corrective supervision, using confidence-gated, temporally aligned successful executions only as residual context rather than replaying their actions.

III. Methodology

As illustrated in Fig. 2, BimanualShift retains frozen unimanual policies [26] and learns lightweight residual coordination through arm-conditioned visual projection, residual skill modulation, and retrieval-conditioned residual refinement.

A. Arm-Conditioned Visual Projection

The first stage converts the shared bimanual workspace into policy-compatible unimanual observations. Feeding the global observation o_t to frozen unimanual policies can cause perceptual interference in close-proximity or arm-crossing scenarios [10], [27]. Unlike soft attention mechanisms that require sufficient task data to learn stable separation [9], we define an explicit arm-conditioned visual projection $\mathcal{P}_k$, where $k \in \{L, R\}$ denotes the left or right arm.

The projection maps the shared observation, language instruction, and proprioceptive state into an arm-specific masked observation:

$$\tilde{o}_{k,t} = \mathcal{P}_k(o_t, l, p_t) = V_t \odot \mathbf{M}_{3D,k,t}, \quad k \in \{L, R\} \tag{1}$$

where $V_t \in \mathbb{R}^{H\times W\times D}$ is the voxelized observation, $\mathbf{M}_{3D,k,t}$ is the arm-specific semantic mask, and $\odot$ denotes element-wise multiplication. Eq. (1) suppresses the opposite arm and task-irrelevant regions before policy inference, making the input closer to the unimanual observation distribution on which π_{uni} was pretrained.

In implementation, $\mathcal{P}_k$ is realized by a semantic mask generation and back-projection pipeline based on Grounded-SAM [28], [29]. Given the instruction l, we extract the target object noun χ_{goal} and construct the text prompt $[\text{robotic arm}, \chi_{\text{goal}}]$. Grounded-SAM detects robotic-arm instances and the task target in image space.

Using calibrated base anchors $\mathbf{b}_L$ and $\mathbf{b}_R$, we assign each candidate arm mask to its nearest anchor:

$$\mathbf{M}_{\text{body},k} = \underset{\mathbf{C}_j \in \mathcal{S}_{\text{arm}}}{\arg\min} \left\| \text{Centroid}(\mathbf{C}_j) - \mathbf{b}_k \right\|_2, \tag{2}$$

where $\mathcal{S}_{\text{arm}}$ denotes the set of candidate masks labeled as robotic arms, and $\mathbf{C}_j$ denotes its j-th candidate mask.

The final 2D mask for arm k is obtained by combining the selected body mask and the target-object mask:

$$\mathbf{M}_{2D,k} = \mathbf{M}_{\text{body},k} \cup \mathbf{M}_{\text{goal}} \tag{3}$$

The 2D mask $\mathbf{M}_{2D,k}$ is then back-projected into the voxel grid using the depth map $\mathcal{D}_t$ and camera intrinsics $\mathbf{K}$, yielding the 3D mask $\mathbf{M}_{3D,k,t}$ used in Eq. (1). As visualized in Fig. 3, this projection filters the interfering arm and background clutter while retaining the acting arm, target object, and local support surface. Consequently, bimanual perception is converted into two decoupled unimanual observation streams, allowing the residual skill modulation stage in Section III-B to focus on coordination rather than resolving visual interference.

B. Residual Skill Modulation

After obtaining arm-specific observations from Eq. (1), BimanualShift coordinates the two frozen unimanual policies through residual skill modulation. As illustrated in Fig. 4, instead of learning a bimanual policy from scratch [2], the Action Generator predicts a low-dimensional residual in a learnable skill basis and adds the projected residual to the frozen unimanual policy output. For each arm $k \in \{L, R\}$, the final action is defined as:

$$a_{k,t} = \pi_{\text{uni}}(\tilde{o}_{k,t}, p_{k,t}) + \mathcal{G}(\upsilon_{k,t}) \tag{4}$$

where π_{uni} is the frozen pretrained unimanual policy, $\tilde{o}_{k,t}$ is the arm-specific observation from Eq. (1), $p_{k,t}$ is the proprioceptive state of arm k, and $\mathcal{G}(\cdot)$ projects the skill representation into the action residual space.

To construct the residual skill representation, we define a learnable skill primitive matrix:

$$\mathcal{Z} = \{\boldsymbol{z}_1, \boldsymbol{z}_2, \ldots, \boldsymbol{z}_N\}, \quad \boldsymbol{z}_i \in \mathbb{R}^{d_{\text{emb}}} \tag{5}$$

where each primitive $\boldsymbol{z}_i$ represents an atomic motion basis [20]. To provide semantic grounding, $\mathcal{Z}$ is initialized from embeddings of atomic action verbs extracted by a frozen CLIP text encoder. In all experiments, we set $N = 16$.

Within the residual skill modulation stage, the Action Generator is implemented as a lightweight Manager Network ϑ_{sm}. Given the arm-specific observation $\tilde{o}_{k,t}$, the bimanual proprioceptive state p_t, the language instruction l, and the gated retrieval context $\mathbf{e}^{\mathrm{ret}}_{k,t}$ defined in Section III-C, it predicts the skill weights $\boldsymbol{\omega}_{k,t}$ and latent compensation vector $\mathbf{h}_{k,t}$:

$$(\boldsymbol{\omega}_{k,t}, \mathbf{h}_{k,t}) = \vartheta_{\mathrm{sm}}(\tilde{o}_{k,t}, p_t, l, \mathbf{e}^{\mathrm{ret}}_{k,t}), \quad \sum_{i=1}^{N} \omega_{i,k,t} = 1. \tag{6}$$

Here, $\boldsymbol{\omega}_{k,t}$ is softmax-normalized and $\mathbf{h}_{k,t} \in \mathbb{R}^{d_{\mathrm{emb}}}$. The retrieval context is projected to the Manager input dimension and is set to zero when the retrieval confidence is below τ_{ret}.

The residual skill representation is then computed as:

$$\upsilon_{k,t} = \sum_{i=1}^{N} \omega_{i,k,t} \boldsymbol{z}_i + \mathbf{h}_{k,t} \tag{7}$$

Here, the weighted skill primitives provide phase-dependent motion structure, while $\mathbf{h}_{k,t}$ compensates for latent deviations induced by local geometric mismatch, contact variation, and inter-arm coordination errors [11]. Through Eq. (4), BimanualShift turns bimanual coordination into residual correction over pretrained unimanual priors, rather than full action generation.

C. Retrieval-Conditioned Residual Refinement

To mitigate compounding errors in long-horizon tasks [12], BimanualShift retrieves successful executions from $\mathcal{M}$ as temporal context for residual generation rather than direct action replay [25]. The fixed memory bank $\mathcal{M}$ contains verified successful rollouts collected by the retrieval-free policy before refinement; corrective and evaluation trajectories are excluded.

Each memory entry stores a task key and a verified successful trajectory, denoted as $(\mathcal{K}_i, \Pi_i^*)$, where $\Pi_i^* = \{(o^*_{i,j}, p^*_{i,j}, a^*_{i,j})\}_{j=1}^{T_i}$ contains the observation, bimanual proprioceptive state, and action at each trajectory step. Since the same instruction may correspond to different initial scenes, the task key jointly encodes the language instruction and initial observation:

$$\mathcal{K}_i = \mathrm{Norm}\left(\left[\mathrm{Norm}(\mathbf{E}_{\mathrm{text}}(l_i)); \mathrm{Norm}(\mathbf{E}_{\mathrm{vision}}(o_0^i))\right]\right), \tag{8}$$

where $\mathrm{Norm}(\mathbf{x}) = \mathbf{x}/(\|\mathbf{x}\|_2 + \epsilon)$, $\mathbf{E}_{\mathrm{text}}$ is the frozen CLIP text encoder, and $\mathbf{E}_{\mathrm{vision}}$ is the frozen visual encoder of the corresponding unimanual backbone. The operator $[\cdot;\cdot]$ denotes feature concatenation.

At episode start, $\mathcal{K}_{\mathrm{query}}$ is computed from (l, o_0) to retrieve the most similar memory entry:

$$\begin{aligned} s_i &= S_{\cos}(\mathcal{K}_{\mathrm{query}}, \mathcal{K}_i), \\ i^* &= \underset{1 \le i \le |\mathcal{M}|}{\arg\max}\, s_i, \qquad c_{\mathrm{ret}} = s_{i^*}. \end{aligned} \tag{9}$$

where $S_{\cos}(\cdot,\cdot)$ denotes cosine similarity and c_{ret} is the resulting retrieval-confidence score. The retrieved execution is accepted only when $c_{\mathrm{ret}} \ge \tau_{\mathrm{ret}}$; otherwise, the retrieval context is disabled. The threshold τ_{ret} is selected on the held-out validation split and remains fixed during evaluation. The retrieved trajectory index i^* is fixed throughout the episode, and subsequent steps perform temporal alignment only within this trajectory.

Because the task-level key in Eq. (8) does not identify the current execution phase, the current bimanual state is further aligned with the retrieved trajectory. Starting from $j_0^* = 1$, the search is restricted to a forward local window:

$$\begin{aligned} \mathcal{W}_t &= \left\{j^*_{t-1}, \ldots, \min(j^*_{t-1} + w, T_{i^*})\right\}, \\ j_t^* &= \underset{j \in \mathcal{W}_t}{\arg\max} S_{\cos}\left(\mathbf{E}_{\mathrm{state}}(o_t, p_t), \mathbf{E}_{\mathrm{state}}(o^*_{i^*,j}, p^*_{i^*,j})\right). \end{aligned} \tag{10}$$

where $\mathbf{E}_{\mathrm{state}}(o_t, p_t) = \mathrm{Norm}([\mathrm{Norm}(\mathbf{E}_{\mathrm{vision}}(o_t)); \mathrm{Norm}(\bar{p}_t)])$, with $\bar{p}_t = (p_t - \mu_p)/(\sigma_p + \epsilon)$ and (μ_p, σ_p) computed from the training set. The monotonic local window accommodates different execution speeds while preventing the matched reference index from moving backward.

The arm-specific retrieval context is then constructed from the matched reference step:

$$\mathbf{e}^{\mathrm{ret}}_{k,t} = \begin{cases} \mathbf{E}_{\mathrm{ref}}\left(\left[\mathbf{E}_{\mathrm{vision}}(o^*_{i^*,j^*_t}); p^*_{i^*,j^*_t}; a^*_{k,i^*,j^*_t}\right]\right), & c_{\mathrm{ret}} \ge \tau_{\mathrm{ret}}, \\ \mathbf{0}, & \text{otherwise}, \end{cases} \tag{11}$$

where $\mathbf{E}_{\mathrm{ref}}$ is a learnable projection layer included in the Manager Network ϑ_{sm} and maps the concatenated reference features to its input dimension. Proprioceptive and action features are standardized using training-set statistics before concatenation. Both arms share the aligned reference index j_t^* while using their corresponding arm-specific actions.

As defined in Eq. (6) of Section III-B, $\mathbf{e}^{\mathrm{ret}}_{k,t}$ conditions both the skill weights $\boldsymbol{\omega}_{k,t}$ and latent compensation vector $\mathbf{h}_{k,t}$, thereby influencing the residual representation in Eq. (7) without directly replaying the retrieved action.

Retrieval provides context rather than supervision. Let $\upsilon^{\mathrm{base}}_{k,t}$ and $\upsilon^{\mathrm{ret}}_{k,t}$ denote the representations produced with zero and valid retrieval contexts, respectively. The retrieval-induced residual increment is defined as:

$$\begin{aligned} \Delta\mathbf{r}^{\mathrm{ret}}_{k,t} &= \mathcal{G}(\upsilon^{\mathrm{ret}}_{k,t}) - \mathrm{sg}\left[\mathcal{G}(\upsilon^{\mathrm{base}}_{k,t})\right], \\ a^{\mathrm{base}}_{k,t} &= \pi_{\mathrm{uni}}(\tilde{o}_{k,t}, p_{k,t}) + \mathcal{G}(\upsilon^{\mathrm{base}}_{k,t}), \end{aligned} \tag{12}$$

where $\mathrm{sg}[\cdot]$ denotes stop-gradient.

When a rollout fails, a successful retry or corrective demonstration provides a local corrective sequence. For each corrective observation $(o_t^{\mathrm{corr}}, p_t^{\mathrm{corr}})$, $a^{\mathrm{base}}_{k,t}$ is recomputed by the zero-context branch at the same state. For a fixed correction horizon H_{corr}, the target increment is defined as

$$\mathbf{r}^*_{k,t} = \begin{cases} a^{\mathrm{corr}}_{k,t} - \mathrm{sg}\left[a^{\mathrm{base}}_{k,t}\right], & t_f \le t < t_f + H_{\mathrm{corr}}, \\ \mathbf{0}, & \text{otherwise}, \end{cases} \tag{13}$$

where zero targets are assigned to steps outside the corrective interval. The correction loss is

$$\mathcal{L}_{\mathrm{corr}} = \mathbb{E}_{c_{\mathrm{ret}} \ge \tau_{\mathrm{ret}}}\left[\sum_{k \in \{L,R\}} \sum_{t} \left\|\Delta\mathbf{r}^{\mathrm{ret}}_{k,t} - \mathbf{r}^*_{k,t}\right\|_2^2\right]. \tag{14}$$

During refinement, $\mathbf{E}_{\mathrm{ref}}$, ϑ_{sm}, and $\mathcal{G}$ are updated, while the pretrained unimanual policy π_{uni} remains frozen. At inference, the final action is still computed by Eq. (4) using $\upsilon^{\mathrm{ret}}_{k,t}$. A concrete example of this retrieval–alignment–refinement process is provided in Section IV-F.

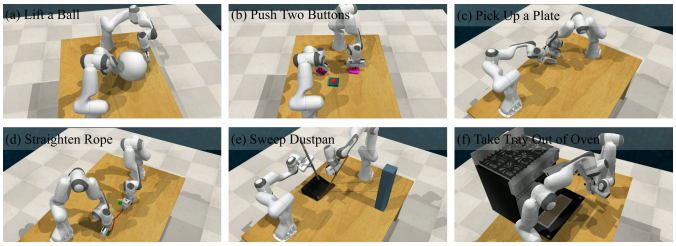


Fig. 5. Bimanual manipulation tasks in RLBench2.

TABLE I
SIMULATION RESULTS AND COMPONENT ABLATIONS ON RLBENCH2

Method	(i) Complex Tasks (30)				(ii) Single-Action Tasks (100)			
	Take Tray Out of Oven	Straighten Rope	Sweep Dustpan	Average	Lift a Ball	Push Two Buttons	Pick Up a Plate	Average
RVT-LF [30]	2.0	4.0	18.0	8.0 ± 1.6	17.0	39.0	3.0	19.7 ± 2.3
AnyBimanual-RVT [9]	4.0	6.0	24.0	11.3 ± 1.8	20.0	42.0	4.0	22.0 ± 2.4
BimanualShift-RVT (Ours)	**6.0**	**9.0**	**30.0**	**15.0 ± 2.0**	**23.0**	**47.0**	4.0	**24.7 ± 2.5**
ACT [2]	0.0	4.0	8.0	4.0 ± 1.2	36.0	4.0	0.0	13.3 ± 2.0
VoxAct-B [14]	4.0	12.0	28.0	14.7 ± 2.1	48.0	15.0	5.0	22.7 ± 2.4
PerAct-LF [26]	3.0	8.0	22.0	11.0 ± 1.9	40.0	10.0	2.0	17.3 ± 2.2
PerAct^2 [13]	**13.0**	10.0	30.0	17.7 ± 2.2	43.0	41.0	4.0	29.3 ± 2.7
AnyBimanual-PerAct [9]	9.0	16.0	41.0	22.0 ± 2.4	**61.0**	58.0	8.0	42.3 ± 3.0
w/o Visual Projection	7.0	16.0	39.0	20.7 ± 2.4	44.0	48.0	7.0	33.0 ± 2.9
w/o Residual Modulation	5.0	12.0	34.0	17.0 ± 2.2	38.0	52.0	6.0	32.0 ± 2.8
w/o Retrieval Refinement	8.0	18.0	43.0	23.0 ± 2.5	52.0	69.0	9.0	43.3 ± 3.1
BimanualShift-PerAct (Ours)	11.0	**27.0**	**57.0**	**31.7 ± 2.8**	60.0	**80.0**	**12.0**	**50.7 ± 3.3**

Note: Mean success rate (%) ± standard deviation across three training seeds.

D. Training Objectives

Training has two stages. First, ϑ_{sm} and $\mathcal{G}$ are behavior-cloned on $\mathcal{D}$ with $\mathbf{e}_{k,t}^{\text{ret}} = \mathbf{0}$. The resulting retrieval-free policy is then used to collect successful rollouts for the fixed memory bank and separate corrective trajectories. During refinement, ϑ_{sm}, including $\mathbf{E}_{\text{ref}}$, and $\mathcal{G}$ are jointly optimized, while the arm-conditioned visual projection module and pretrained unimanual policy π_{uni} remain frozen.

We decompose the target action into translation a_{trans} and axis-angle rotation θ_{axis}:

$$\mathcal{L}_{\text{BC}} = \mathbb{E}_{(s,a)\sim\mathcal{D}}\left[\|a_{\text{trans}} - \hat{a}_{\text{trans}}\|_2^2 + \beta\|\theta_{\text{axis}} - \hat{\theta}_{\text{axis}}\|_2^2\right], \tag{15}$$

where $\hat{a}_{\text{trans}}$ and $\hat{\theta}_{\text{axis}}$ denote the expert translation and rotation targets, and β balances the two terms.

During refinement, $\mathcal{L}_{\text{BC}}$ uses zero retrieval context, while $\mathcal{L}_{\text{corr}}$ follows Eqs. (12)–(14). The final objective is:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{BC}} + \lambda_{\text{corr}}\mathcal{L}_{\text{corr}}. \tag{16}$$

Only the lightweight residual coordination components are updated by this objective.

IV. EXPERIMENTAL EVALUATION AND RESULTS

A. Simulation in RLBench2

We evaluate BimanualShift on six representative tasks from RLBench2 [13], a bimanual extension of RLBench [31], using two Franka Panda arms and six noise-free 256×256 RGB-D cameras.

Baselines: We compare BimanualShift with representative bimanual manipulation baselines, including voxel-based methods PerAct^2 [13] and VoxAct-B [14], the trajectory-prediction method ACT [2], Leader-Follower variants RVT-LF [30] and PerAct-LF [26], and AnyBimanual [9], the closest unimanual-to-bimanual transfer baseline.

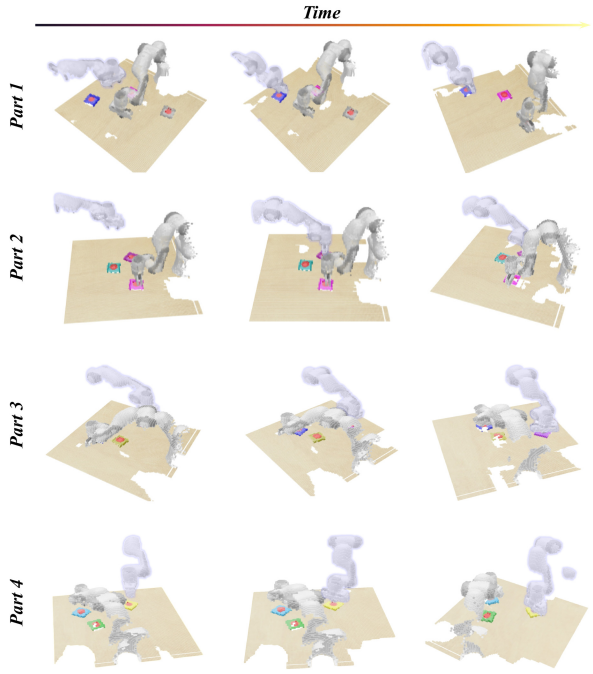


Fig. 6. Randomization examples of Push Two Buttons. Object colors and target positions vary across executions, and the shown images correspond to the generated left- and right-arm masks.

Simulation Setup: To reflect real-world data-collection costs, we split the six tasks into complex 30-demo tasks (Take Tray Out of Oven, Straighten Rope, Sweep Dustpan) and simpler 100-demo tasks (Lift a Ball, Push Two Buttons, Pick Up a Plate) (see Fig. 5). All tasks include randomization in object color, size, quantity, placement, and category (see Fig. 6). We evaluate each method over 100 episodes per task on a single NVIDIA RTX 4090 GPU and report average success rates. All unimanual policies are initialized from checkpoints pretrained on 18 RLBench tasks [31].

Implementation Details: Baselines are trained on two NVIDIA RTX A10 GPUs. BimanualShift keeps the pretrained unimanual backbones frozen and optimizes only the lightweight adaptation modules. BimanualShift-PerAct uses the public PerAct checkpoint pretrained on 18 RLBench tasks. We use LAMB with a learning rate of 5×10^{-4}, batch size 4, SE(3) augmentation, sparse keyframes, and motion planning for full-trajectory execution. No evaluation episode contributes to memory updates or corrective supervision. For each task, we autonomously collect 10 successful policy rollouts for the memory bank and use 5 additional failure-correction trajectories only for residual refinement. We set the retrieval threshold to $\tau_{\text{ret}} = 0.80$, the local alignment window to $w = 2$, and the correction horizon to $H_{\text{corr}} = 2$ keyframes for all experiments. In simulation, t_f is automatically identified from the first violated subtask condition, while in real-world experiments it is manually annotated.

Ablation Setup: To analyze the contribution of each component, we further evaluate three ablated variants of BimanualShift-PerAct: w/o Visual Projection, w/o Residual Modulation, and w/o Retrieval Refinement. All ablation variants use the same pretrained frozen PerAct backbone and are

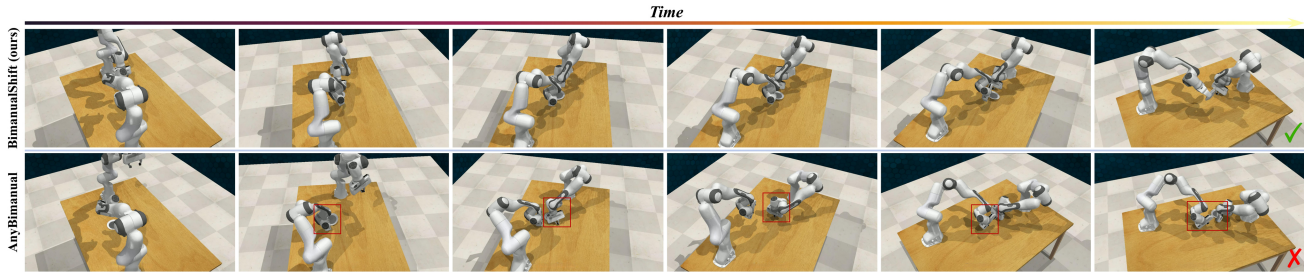


Fig. 7. Comparison between BimanualShift and AnyBimanual on the Pick Up a Plate task.

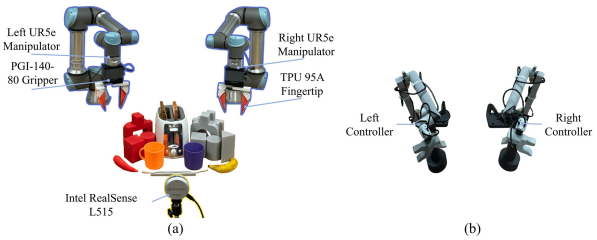


Fig. 8. Real-world setup. (a) Dual-UR5e platform with an Intel RealSense L515 camera. (b) GELLO teleoperation device.

TABLE II
REAL-WORLD TASK EVALUATION

Tier	Task	Episodes	Keyframes	BimanualShift-PerAct	BimanualShift-DP	DP [3]	π_0-LoRA [4]
Coordination	Toasting Completion	20	6	100.0 ± 0.0	95.0 ± 4.9	30.0 ± 10.2	65.0 ± 3.5
	Vegetable Sorting	20	4	**100.0 ± 0.0**	**100.0 ± 0.0**	40.0 ± 11.0	60.0 ± 6.1
Collaboration	Flower Arrangement	20	5	85.0 ± 7.5	70.0 ± 9.5	10.0 ± 6.5	40.0 ± 6.1
	Pouring Water	20	4	75.0 ± 8.5	35.0 ± 8.0	0.0 ± 0.0	20.0 ± 7.1
	Quilt Folding	20	5	90.0 ± 9.5	75.0 ± 8.0	10.0 ± 6.5	45.0 ± 7.9
	Toaster Activation	20	6	80.0 ± 8.5	45.0 ± 9.5	5.0 ± 5.0	30.0 ± 7.9
Precision	Block Threading	20	4	95.0 ± 5.0	65.0 ± 12.0	0.0 ± 0.0	20.0 ± 11.2
	Cable Routing	20	5	55.0 ± 15.5	35.0 ± 12.5	0.0 ± 0.0	10.0 ± 6.1
–	Average	–	–	85.0 ± 4.5	65.0 ± 6.0	11.9 ± 3.5	36.3 ± 6.6

trained with the same demonstrations as the full model.

The results are summarized in Table I. Under the 30-demonstration complex-task setting, BimanualShift-PerAct achieves the highest average success rate of 31.7%, outperforming PerAct^2 (17.7%), VoxAct-B (14.7%), PerAct-LF (11.0%), and AnyBimanual-PerAct (22.0%). Under the 100-demonstration setting, it reaches the best average success rate of 50.7%, exceeding PerAct^2 (29.3%), VoxAct-B (22.7%), and AnyBimanual-PerAct (42.3%). Although PerAct^2 and AnyBimanual-PerAct perform better on Take Tray Out of Oven and Lift a Ball, respectively, BimanualShift-PerAct achieves stronger overall performance on tasks involving deformable-object interaction, close-proximity coordination, or target disambiguation, such as Straighten Rope, Sweep Dustpan, Push Two Buttons, and Pick Up a Plate. The ablation results show that removing Visual Projection, Residual Modulation, or Retrieval Refinement reduces the average success rates to 20.7%/33.0%, 17.0%/32.0%, and 23.0%/43.3%, respectively, confirming their complementary contributions. As shown in Fig. 7, BimanualShift reduces target-assignment ambiguity by better separating arm-specific workspaces.

B. Real-World Setup

We evaluate BimanualShift on a real dual-UR5e platform with PGI-140 grippers, custom TPU 95A compliant fingertips, and a front-mounted Intel RealSense L515 camera (see Fig. 8). The system runs on Ubuntu 20.04 with ROS1. Demonstrations are collected via GELLO teleoperation [32], which maps

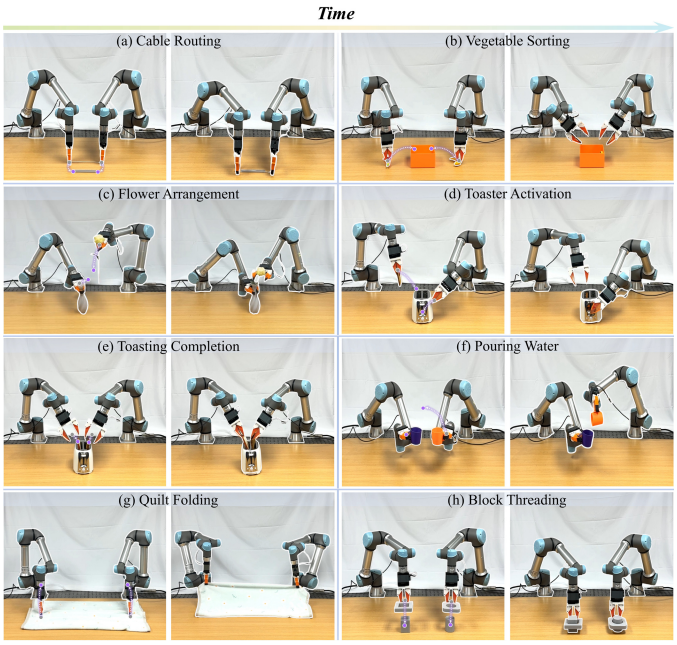


Fig. 9. Real-world bimanual manipulation tasks.

human bimanual motions to 6-DoF end-effector poses. After hand–eye calibration with easy_handeye and ArUco markers, RGB images are cropped to 256×256 and keyframes are extracted. At deployment, BimanualShift predicts action chunks $a_{t:t+H}$ from real-time observations and executes target poses using MoveIt.

C. Core Performance and Robustness in Real-World

To evaluate BimanualShift across different backbones, we instantiate it with PerAct and Diffusion Policy (DP) [3], [13], both pretrained on separate unimanual data. We compare them with End-to-End DP trained from scratch and π_0-LoRA fine-tuned on the same real-world bimanual demonstrations. For each of the eight tasks, we collect 15 demonstrations; BimanualShift adapts lightweight coordination modules while keeping the unimanual policies frozen. All methods are evaluated on the real platform shown in Fig. 9.

The results are summarized in Table II. End-to-End DP achieves only 40% and 30% success on Vegetable Sorting and Toasting Completion, respectively, whereas BimanualShift-DP increases these to 100% and 95%. π_0-LoRA improves over End-to-End DP, achieving 60% and 65% on the two coordination tasks and an average success rate of 36.3%, but remains less effective on contact-rich and precision-critical

TABLE III
GENERALIZATION RESULTS ON THE BLOCK THREADING TASK

Condition	Method	Approach	Alignment	Insertion	Average	RDR[1]	Condition	Method	Approach	Alignment	Insertion	Average	RDR[1]
Standard	PerAct-LF	90.0	70.0	30.0	63.3	-	Left–Right Exchange	PerAct-LF	55.0	30.0	5.0	30.0	52.6
	BimanualShift-PerAct	**100.0**	**100.0**	**95.0**	**98.3**	-		**BimanualShift-PerAct**	**90.0**	**80.0**	**75.0**	**81.7**	**16.9**
Unseen Color	PerAct-LF	80.0	60.0	20.0	53.3	15.8	Unseen Background	PerAct-LF	60.0	35.0	10.0	35.0	44.7
	BimanualShift-PerAct	**100.0**	**95.0**	**90.0**	**95.0**	**3.4**		**BimanualShift-PerAct**	**85.0**	**75.0**	**70.0**	**76.7**	**22.0**
Unseen Shape	PerAct-LF	70.0	45.0	15.0	43.3	31.6	Extreme Glare	PerAct-LF	75.0	50.0	15.0	46.7	26.2
	BimanualShift-PerAct	**95.0**	**85.0**	**80.0**	**86.7**	**11.8**		**BimanualShift-PerAct**	**100.0**	**75.0**	**55.0**	**76.7**	**22.0**
Lighting Change	PerAct-LF	65.0	40.0	10.0	38.3	39.5	Camera Perturbation	PerAct-LF	40.0	15.0	5.0	20.0	68.4
	BimanualShift-PerAct	**90.0**	**80.0**	**75.0**	**81.7**	**16.9**		**BimanualShift-PerAct**	**90.0**	**65.0**	**50.0**	**68.3**	**30.5**

[1] RDR: relative degradation rate with respect to the standard-setting average of the same method.

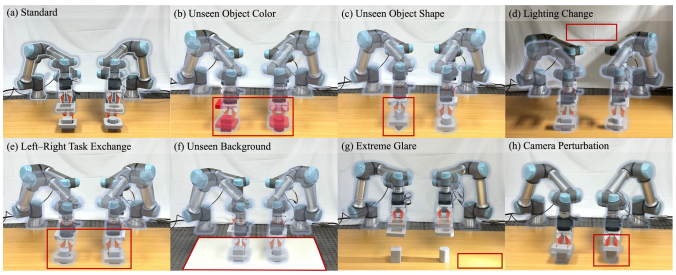


Fig. 10. Seven generalization scenarios for the Block Threading task.

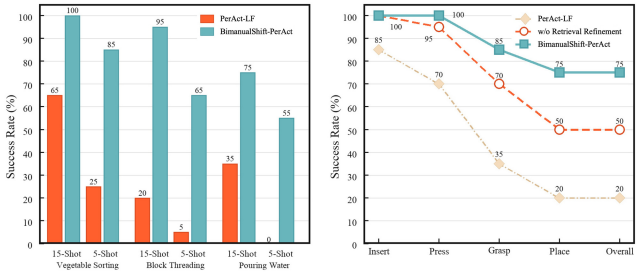


Fig. 11. Data scarcity and continuous learning analysis. (a) Few-shot performance on representative real-world tasks. (b) Segment-wise performance on the long-horizon task.

tasks. BimanualShift-PerAct performs best overall, achieving 95% on Block Threading and 75%/55% on Pouring Water and Cable Routing, respectively. These results show that generic VLA pretraining provides useful manipulation priors, while explicit reuse and coordination of unimanual policies remain important for precise bimanual manipulation.

D. Generalization in Real-World

To evaluate real-world generalization, we study the challenging Block Threading task using the best-performing BimanualShift-PerAct. As shown in Fig. 10, we consider seven settings: unseen object color, unseen object shape, lighting change, left–right task exchange, unseen background, extreme glare, and camera perturbation. Each setting is evaluated over 20 trials, with stage-wise success rates reported for Approach, Alignment, and Insertion, together with average performance and relative degradation.

As shown in Table III, BimanualShift-PerAct consistently outperforms PerAct-LF. Under the standard setting, it achieves 98.3% average success and 95.0% insertion success, compared with 63.3% and 30.0% for PerAct-LF. Its average success remains 95.0%, 86.7%, 81.7%, and 76.7% under the four appearance changes. Under left–right exchange, extreme glare, and camera perturbation, it achieves 81.7%, 76.7%, and 68.3%, respectively, versus 30.0%, 46.7%, and 20.0% for PerAct-LF. These results demonstrate improved robustness to appearance, task configuration, illumination, and camera variations.

E. Performance in Data-Scarce Regimes

To evaluate performance under extremely limited demonstrations, we select three representative real-world tasks: Vegetable Sorting, Block Threading, and Pouring Water. BimanualShift-PerAct and PerAct-LF are trained with only 15 or 5 demonstrations per task and evaluated over 20 trials.

As shown in Fig. 11(a), BimanualShift-PerAct consistently outperforms PerAct-LF in both 15-shot and 5-shot settings. In the 15-shot setting, BimanualShift-PerAct achieves 100.0%, 95.0%, and 75.0% success on Vegetable Sorting, Block Threading, and Pouring Water, respectively, while PerAct-LF drops to 65.0%, 20.0%, and 35.0%. Under the more challenging 5-shot setting, BimanualShift-PerAct still maintains 85.0%, 65.0%, and 55.0% success, whereas PerAct-LF degrades to 25.0%, 5.0%, and 0.0%. These results indicate that frozen unimanual priors and lightweight adaptation enable more data-efficient bimanual skill transfer under severe data scarcity.

F. Continuous Learning Capability

To evaluate retrieval-conditioned residual refinement in long-horizon execution, we construct a long-horizon task after the model has learned Toasting Completion and Toaster Activation. The task consists of four sequential actions: Insert, Press, Grasp, and Place (see Fig. 12). We fine-tune BimanualShift-PerAct with only 5 demonstrations for the novel skill and evaluate it over 20 trials, comparing against the variant w/o Retrieval Refinement and PerAct-LF. For this task, the most relevant rollout is retrieved from the fixed memory bank of 10 successful rollouts and confidence-filtered at episode start. During Insert, Press, Grasp, and Place, the locally aligned retrieval context conditions the coordination residual.

As shown in Fig. 11(b), BimanualShift-PerAct achieves 100.0% success on the previously learned Insert and Press stages, indicating effective retrieval of prior residual contexts. It further maintains 85.0% and 75.0% success on the Grasp and Place stages, outperforming w/o Retrieval Refinement by 15 and 25 percentage points, respectively. Compared with

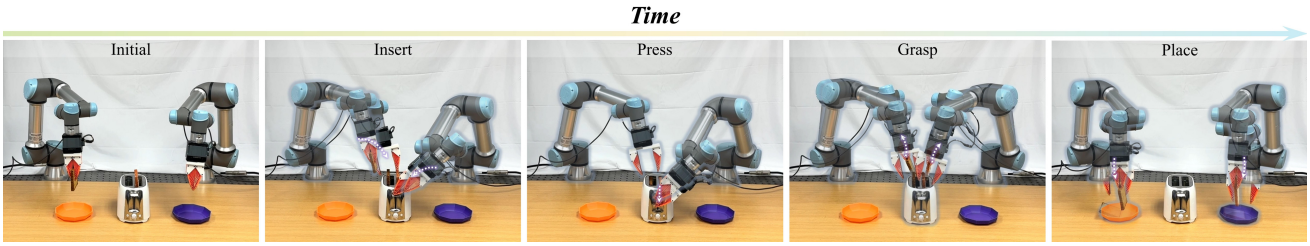


Fig. 12. Long-horizon task composed of four consecutive sub-actions: Insert, Press, Grasp, and Place.

PerAct-LF, which drops sharply after Press and achieves only 20.0% overall success, BimanualShift-PerAct reaches 75.0% overall success, demonstrating that retrieval-conditioned residual refinement improves long-horizon execution under limited demonstrations.

V. Conclusion

This letter presented BimanualShift, an arm-conditioned residual skill transfer framework that reformulates unimanual-to-bimanual transfer as residual coordination over frozen unimanual policy priors. Instead of training a full bimanual policy from scratch, BimanualShift aligns shared observations to policy-compatible arm-specific inputs, re-parameterizes dual-arm coordination as lightweight residual modulation, and reuses successful executions as residual contexts for long-horizon correction. Experiments in RLBench2 and on a real dual-arm platform show that BimanualShift improves few-shot transfer, generalization, and robustness, achieving a 31.7% average success rate in the 30-demonstration complex-task setting and 85.0% average success across eight real-world tasks.

References

[1] J. Ye *et al.*, "Towards Industry 5.0: Emerging trends in dual-arm human–robot collaboration," *Biomimetic Intell. Robot.*, vol. 6, no. 2, Art. no. 100332, 2026.

[2] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *Proc. Robot.: Sci. Syst.*, 2023.

[3] C. Chi *et al.*, "Diffusion policy: Visuomotor policy learning via action diffusion," *Int. J. Robot. Res.*, vol. 44, pp. 1684–1704, 2025.

[4] K. Black *et al.*, "π_0: A vision-language-action flow model for general robot control," in *Proc. Robot.: Sci. Syst.*, 2025.

[5] K. Black *et al.*, "$\pi_{0.5}$: A vision-language-action model with open-world generalization," *Proc. Mach. Learn. Res.*, vol. 305, pp. 17–40, 2025.

[6] A. Amin *et al.*, "$\pi^*_{0.6}$: A VLA that learns from experience," *arXiv preprint arXiv:2511.14759*, 2025.

[7] D. Ghosh *et al.*, "Octo: An open-source generalist robot policy," in *Proc. Robot.: Sci. Syst. (RSS)*, 2024.

[8] M. J. Kim *et al.*, "OpenVLA: An open-source vision-language-action model," *Proc. Mach. Learn. Res.*, vol. 270, pp. 2679–2713, 2025.

[9] G. Lu, T. Yu, H. Deng, S. S. Chen, Y. Tang, and Z. Wang, "AnyBimanual: Transferring unimanual policy for general bimanual manipulation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2025, pp. 13662–13672.

[10] F. Krebs and T. Asfour, "A bimanual manipulation taxonomy," *IEEE Robot. Autom. Lett.*, vol. 7, no. 4, pp. 11031–11038, 2022.

[11] A. Sintov, S. Macenski, A. Borum, and T. Bretl, "Motion planning for dual-arm manipulation of elastic rods," *IEEE Robot. Autom. Lett.*, vol. 5, no. 4, pp. 6065–6072, 2020.

[12] M. Drolet *et al.*, "A comparison of imitation learning algorithms for bimanual manipulation," *IEEE Robot. Autom. Lett.*, vol. 9, no. 10, pp. 8579–8586, 2024.

[13] M. Grotz, M. Shridhar, Y.-W. Chao, T. Asfour, and D. Fox, "PerAct2: Benchmarking and learning for robotic bimanual manipulation tasks," in *Proc. CoRL Workshop Whole-Body Control Bimanual Manipulation*, 2024.

[14] I.-C. A. Liu, S. He, D. Seita, and G. S. Sukhatme, "VoxAct-B: Voxel-based acting and stabilizing policy for bimanual manipulation," *Proc. Mach. Learn. Res.*, vol. 270, pp. 4354–4370, 2025.

[15] Y. Hu *et al.*, "Video prediction policy: A generalist robot policy with predictive visual representations," *Proc. Mach. Learn. Res.*, vol. 267, pp. 24328–24346, 2025.

[16] Y. Fan *et al.*, "Tool-Action Comprehension: A cross-morphology imitation learning framework for bimanual manipulation," *IEEE/ASME Trans. Mechatronics*, vol. 31, no. 4, pp. 4014–4025, 2026.

[17] X. Li *et al.*, "ManipLLM: Embodied multimodal large language model for object-centric robotic manipulation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 18061–18070.

[18] M. Pan, J. Zhang, T. Wu, Y. Zhao, W. Gao, and H. Dong, "OmniManip: Towards general robotic manipulation via object-centric interaction primitives as spatial constraints," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2025, pp. 17359–17369.

[19] D. Dwibedi, Y. Aytar, J. Tompson, P. Sermanet, and A. Zisserman, "Temporal cycle-consistency learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 1801–1810.

[20] M. Xu, Z. Xu, C. Chi, M. Veloso, and S. Song, "XSkill: Cross embodiment skill discovery," in *Proc. Conf. Robot Learn.*, 2023, pp. 3536–3555.

[21] H. Im, E. Jeong, A. Kolobov, J. Fu, and Y. Lee, "TwinVLA: Data-efficient bimanual manipulation with twin single-arm vision-language-action models," in *Proc. Int. Conf. Learn. Represent.*, 2026.

[22] M. Song, X. Deng, J. Wei, D. Jiang, L. Nie, and W. Guan, "EnergyAction: Unimanual to bimanual composition with energy-based models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2026, pp. 20845–20855.

[23] L. Ankile, A. Simeonov, I. Shenfeld, M. Torne, and P. Agrawal, "From imitation to refinement—Residual RL for precise assembly," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2025, pp. 3164–3171.

[24] Y. Zhu, Z. Ou, X. Mou, and J. Tang, "Retrieval-augmented embodied agents," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 17985–17995.

[25] N. Di Palo and E. Johns, "DINOBot: Robot manipulation via retrieval and alignment with vision foundation models," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2024, pp. 2798–2805.

[26] M. Shridhar, L. Manuelli, and D. Fox, "Perceiver-actor: A multi-task transformer for robotic manipulation," in *Proc. Conf. Robot Learn.*, 2023, pp. 785–799.

[27] Y. Fan *et al.*, "Hemispheric Diffusion: A compositional generative policy for coordinated bimanual manipulation," *IEEE Robot. Autom. Lett.*, early access, 2026, doi: 10.1109/LRA.2026.3746919.

[28] S. Liu *et al.*, "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," in *Proc. Eur. Conf. Comput. Vis.*, 2024, pp. 38–55.

[29] A. Kirillov *et al.*, "Segment anything," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 4015–4026.

[30] A. Goyal, J. Xu, Y. Guo, V. Blukis, Y.-W. Chao, and D. Fox, "RVT: Robotic view transformer for 3D object manipulation," in *Proc. Conf. Robot Learn.*, 2023, pp. 694–710.

[31] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison, "RLBench: The robot learning benchmark & learning environment," *IEEE Robot. Autom. Lett.*, vol. 5, pp. 3019–3026, 2020.

[32] P. Wu, Y. Shentu, Z. Yi, X. Lin, and P. Abbeel, "GELLO: A general, low-cost, and intuitive teleoperation framework for robot manipulators," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2024, pp. 12156–12163.